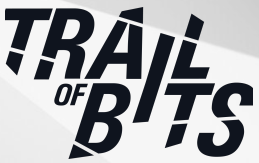

Offchain Reward Distributor Fixes

Security Assessment (Summary Report)

June 16, 2026

Prepared for:
Harry Kalodner, Steven Goldfeder, and Ed Felten
Offchain Labs

Prepared by: **Gustavo Grieco, Simone Monica, Jaime Iglesias, and Nicolas Donboly**

Table of Contents

Project Summary

Contact Information

The following project manager was associated with this project:

Mary O'Brien, Project Manager
mary.obrien@trailofbits.com

The following engineering director was associated with this project:

Benjamin Samuels, Engineering Director, Blockchain
benjamin.samuels@trailofbits.com

The following consultants were associated with this project:

Gustavo Grieco, Consultant
gustavo.grieco@trailofbits.com

Simone Monica, Consultant
simone.monica@trailofbits.com

Jaime Iglesias, Consultant
jaime.iglesias@trailofbits.com

Nicolas Donboly, Consultant
nicolas.donboly@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
April 9, 2025	Delivery of report draft
April 9, 2025	Report readout meeting
April 18, 2025	Delivery of final summary report
June 10, 2026	Delivery of draft extended summary report
June 16, 2026	Delivery of final summary report

Executive Summary

Engagement Overview

Offchain Labs engaged Trail of Bits to review the security of the changes made to the reward distributor contract to support handling of ERC-20 tokens specified at deployment (e.g., WETH). These changes correspond to PR #44 (`fafc251`).

The commits in scope involve a number of changes that allow reward distributor contracts to use ERC-20 tokens to properly reimburse the data availability committee members. Essentially, a set of addresses are paid based on a formula that is computed on-chain (the formula was not modified in the changes in scope). Only the infrastructure smart contract–related changes were in scope, so testing and deployment code was excluded.

A team of four consultants conducted the review from April 7 to April 8, 2025, for a total of four engineer-days of effort. With full access to source code and documentation, we performed a manual review of the code in scope.

Finally, on June 3, 2026, we reviewed some additional changes made to the reward distributor as part of PR #1 (`a64cc11`) for a total of two additional engineer days of effort. These changes fix a specific interaction between the reward distributor router and the Arbitrum L2 standard ERC20 gateway, and the OP Standard Bridge on OptimismMintableERC20 tokens.

These two contracts, instead of consuming the allowance (as previously assumed), burn the tokens, leading to denial-of-service when the reward distributor tries to set the allowance for the next distribution.

Observations and Impact

This engagement revealed a single Information issue in the code in scope. Additionally, we provide some recommendations for improving the code quality in the Code Quality Recommendations appendix.

Recommendations

Based on the security review, Trail of Bits recommends that Offchain Labs take the following step:

- Review the finding presented in this report and consider taking action on it.
- Review the items in the Code Quality Recommendations appendix and consider taking action on each one.

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	Severity
1	Zero allowance is incompatible with some ERC-20 tokens	Undefined Behavior	**Informational**

Detailed Findings

1. Zero allowance is incompatible with some ERC-20 tokens

Severity: **Informational**	Difficulty: **Medium**
Type: Undefined Behavior	Finding ID: TOB-REWARD-1
Target: `feeRouter/ArbChildToParentRewardRouter.sol`, `feeRouter/OpChildToParentRewardRouter.sol`	

Description

PR #1 (a64cc11) introduces a number of changes to prevent reward distributions from entering a denial of service (DoS) state in certain situations. Particularly when interacting with contracts that do not consume the allowance but instead burn the tokens of the caller.

The proposed fix, basically resets the allowance (by setting it to zero) before a new allowance is set for the distribution, preventing the DoS scenario from arising.

```
function _sendToken(uint256 amount) internal override {
    // get gateway from gateway router
    address gateway = childChainGatewayRouter.getGateway(parentChainTokenAddress);

    // approve for transfer
    IERC20(childChainTokenAddress).safeApprove(gateway, 0);
```

Figure 1.1: part of the _sendToken function in ArbChildToParentRewardRouter.sol#78-L83

The same fix, as seen below, is applied to both the Gateway and Optimism reward routers.

```
function _sendToken(uint256 amount) internal override {
    // approve for transfer
    // (not actually necessary for non-native tokens)
    IERC20(childChainTokenAddress).safeApprove(address(opStandardBridge), 0);
```

Figure 1.2: part of the _sendToken function in OpChildToParentRewardRouter.sol#L51-L54

While the fix is correct, there are certain tokens (such as BNB) that will revert when an allowance of zero is attempted making the fix ineffective for them.

Recommendations

Short term, consider whether the aforementioned edge-case is acceptable. In the case that it is not then consider alternative mechanism to support said tokens. For example, making a zero approval during deployment (i.e in the constructor) such that the deployment of the router fails if the token is unsupported, that way an alternative router contract could be used (e.g one that doesn't have the zero approval).

Long term, whenever a contract is interacting with arbitrary tokens, consider the possible edge cases that could arise, and document how the protocol should behave.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	**Description**
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	**Description**
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	**Description**
Not Applicable	This issue is of informational severity and does not pose an immediate risk, so difficulty does not apply.
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Recommendations

The following is a list of findings that were not identified as immediate security issues but may warrant further investigation.

- **Consider adding documentation on the preconditions for the funds distribution.** For example, the following should be documented:
    - When new recipients are added, the list is not checked for duplicate recipients.
    - When new recipients are added, their assigned weights are not checked to ensure they are nonzero.

  While it is assumed that these checks are performed off-chain and the admin is trusted, the contract code could include some documentation about them.
- **Consider correcting the following typos:**
    - `recieve` should be changed to `receive`.
        - `src/RewardDistributor.sol#L21`
        - `src/FeeRouter/ChildToParentRewardRouter.sol#L10`
    - `OwnerRecieved` should be changed to `OwnerReceived`.
        - `src/RewardDistributor.sol#L43`
        - `src/RewardDistributor.sol#L151`
    - `RecipientRecieved` should be changed to `RecipientReceived`.
        - `src/RewardDistributor.sol#L46`
        - `src/RewardDistributor.sol#L139`
    - `reminder` should be changed to `remainder`.
        - `src/RewardDistributor.sol#L112`
    - `committment` should be changed to `commitment`.
        - `src/RewardDistributor.sol#L183`
        - `src/RewardDistributor.sol#L187`

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at https://github.com/trailofbits/publications, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review projects, supporting client organizations in the technology, defense, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, and Zoom.

Trail of Bits also operates a center of excellence with regard to blockchain security. Notable projects include audits of Algorand, Bitcoin SV, Chainlink, Compound, Ethereum 2.0, MakerDAO, Matic, Uniswap, Web3, and Zcash.

To keep up to date with our latest news and announcements, please follow @trailofbits on Twitter and explore our public repositories at https://github.com/trailofbits. To engage us directly, visit our "Contact" page at https://www.trailofbits.com/contact, or email us at info@trailofbits.com.

Trail of Bits, Inc.
228 Park Ave S #80688
New York, NY 10003
https://www.trailofbits.com
info@trailofbits.com

Notices and Remarks

Copyright and Distribution

Test Coverage Disclaimer

All activities undertaken by Trail of Bits in association with this project were performed in accordance with a statement of work and agreed upon project plan.

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

Trail of Bits uses automated testing techniques to rapidly test the controls and security properties of software. These techniques augment our manual security review work, but each has its limitations: for example, a tool may not generate a random edge case that violates a property or may not fully complete its analysis during the allotted time. Their use is also limited by the time and resource constraints of a project.